Spotify Data Science Project:

Tuinkal Sharma

2025-12-04

```r
# Load necessary libraries
library(readr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(ggplot2)
library(tidyr)
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.5.2
```

```
## Loading required package: lattice
```

```r
library(e1071) # For Naive Bayes
```

```
## Warning: package 'e1071' was built under R version 4.5.2
```

```r
# Load the dataset (Assuming the file is in the same directory or accessible)
df_original <- read_csv("spotify_data clean.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 8573 Columns: 15
```

```
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (7): track_id, track_name, artist_name, artist_genres, album_id, album_...
## dbl  (6): track_number, track_popularity, artist_popularity, artist_follower...
## lgl  (1): explicit
## date (1): album_release_date
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Display initial structure and first few rows
cat("Dataset Structure:\n")
```

```
## Dataset Structure:
```

```
print(str(df_original))
```

```
## spc_tbl_ [8,573 x 15] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ track_id          : chr [1:8573] "3EJS5LyekDim1Tf5rBFmZl" "1oQW6G2ZiwMuHqlPpP27DB" "7mdkjzoIYlf1r
##  $ track_name        : chr [1:8573] "Trippy Mane (ft. Project Pat)" "OMG!" "Hard 2 Find" "Still Get
##  $ track_number      : num [1:8573] 4 1 1 8 2 1 1 1 1 1 ...
##  $ track_popularity  : num [1:8573] 0 0 4 30 0 2 20 27 16 33 ...
##  $ explicit          : logi [1:8573] TRUE TRUE TRUE TRUE TRUE FALSE ...
##  $ artist_name       : chr [1:8573] "Diplo" "Yelawolf" "Riff Raff" "Diplo" ...
##  $ artist_popularity : num [1:8573] 77 64 48 77 48 46 30 49 6 48 ...
##  $ artist_followers  : num [1:8573] 2812821 2363438 193302 2813710 8682 ...
##  $ artist_genres     : chr [1:8573] "moombahton" "country hip hop, southern hip hop" "N/A" "moombaht
##  $ album_id          : chr [1:8573] "5QRFnGnBeMGePBKF2xTz5z" "4SUmmwnv0xTjRcLdjczGg2" "3E3zEAL8gUYWa
##  $ album_name        : chr [1:8573] "d00mscrvll, Vol. 1" "OMG!" "Hard 2 Find" "d00mscrvll, Vol. 1" .
##  $ album_release_date: Date[1:8573], format: "2025-10-31" "2025-10-31" ...
##  $ album_total_tracks: num [1:8573] 9 1 1 9 2 3 1 2 2 1 ...
##  $ album_type        : chr [1:8573] "album" "single" "single" "album" ...
##  $ track_duration_min: num [1:8573] 1.55 3.07 2.55 1.69 2.39 2.76 4.12 2.53 2.92 2.4 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   track_id = col_character(),
##   ..   track_name = col_character(),
##   ..   track_number = col_double(),
##   ..   track_popularity = col_double(),
##   ..   explicit = col_logical(),
##   ..   artist_name = col_character(),
##   ..   artist_popularity = col_double(),
##   ..   artist_followers = col_double(),
##   ..   artist_genres = col_character(),
##   ..   album_id = col_character(),
##   ..   album_name = col_character(),
##   ..   album_release_date = col_date(format = ""),
##   ..   album_total_tracks = col_double(),
##   ..   album_type = col_character(),
##   ..   track_duration_min = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
## NULL
```

```r
cat("\n First 6 Observations:\n")
```

```
##
##  First 6 Observations:
```

```r
print(head(df_original))
```

```
## # A tibble: 6 x 15
##   track_id         track_name track_number track_popularity explicit artist_name
##   <chr>            <chr>             <dbl>            <dbl> <lgl>    <chr>
## 1 3EJS5LyekDim1Tf~ Trippy Ma~            4                0 TRUE     Diplo
## 2 1oQW6G2ZiwMuHql~ OMG!                  1                0 TRUE     Yelawolf
## 3 7mdkjzoIYlf1rx9~ Hard 2 Fi~            1                4 TRUE     Riff Raff
## 4 67rW0Zl7oB3qEpD~ Still Get~            8               30 TRUE     Diplo
## 5 15xptTfRBrjsppW~ ride me l~            2                0 TRUE     Rumelis
## 6 4ccpCcZYseq8VrP~ BLEED                 1                2 FALSE    Minzie
## # i 9 more variables: artist_popularity <dbl>, artist_followers <dbl>,
## #   artist_genres <chr>, album_id <chr>, album_name <chr>,
## #   album_release_date <date>, album_total_tracks <dbl>, album_type <chr>,
## #   track_duration_min <dbl>
```

```r
# Check dimensions
cat("\n Dimensions (Rows, Columns):\n")
```

```
##
##  Dimensions (Rows, Columns):
```

```r
print(dim(df_original))
```

```
## [1] 8573   15
```

```r
set.seed(42) # For reproducibility

# 1. Introduce Missing Values (NA)
df_corrupted <- df_original
df_corrupted$track_popularity[sample(1:nrow(df_corrupted), 50)] <- NA
df_corrupted$track_duration_min[sample(1:nrow(df_corrupted), 30)] <- NA

# 2. Introduce an Outlier
df_corrupted$track_duration_min[sample(1:nrow(df_corrupted), 1)] <- 1000

# 3. Introduce Duplicates
df_corrupted <- bind_rows(df_corrupted, head(df_corrupted, 10))

# Check missing percentages
cat("Missing Value Check (Percentage):\n")
```

```
## Missing Value Check (Percentage):
```

```r
sapply(df_corrupted, function(col) sum(is.na(col)) / length(col) ) %>% round(digits=3)
```

```
##           track_id         track_name       track_number   track_popularity
##              0.000              0.000              0.000              0.006
##           explicit        artist_name  artist_popularity   artist_followers
##              0.000              0.000              0.000              0.000
##      artist_genres           album_id         album_name album_release_date
##              0.000              0.000              0.000              0.001
## album_total_tracks         album_type track_duration_min
##              0.001              0.000              0.004
```

```r
#Remove duplicates and use Median Imputation for missing numerical values, as the median is robust agai

# 1. Handle Duplicates
df_clean <- df_corrupted %>%
  distinct()

# 2. Handle Missing Values (NA)
median_pop <- median(df_clean$track_popularity, na.rm = TRUE)
df_clean$track_popularity <- replace_na(df_clean$track_popularity, median_pop)
df_clean$track_duration_min <- replace_na(df_clean$track_duration_min, median(df_clean$track_duration_mi

# 3. Handle Outliers (IQR Capping for 'track_duration_min')
iqr_dur <- IQR(df_clean$track_duration_min)
q3_dur <- quantile(df_clean$track_duration_min, 0.75)
upper_bound_dur <- q3_dur + 1.5 * iqr_dur

df_clean$track_duration_min <- ifelse(
  df_clean$track_duration_min > upper_bound_dur,
  upper_bound_dur,
  df_clean$track_duration_min
)

# 4. Conversion
df_clean$explicit <- as.factor(df_clean$explicit)
df_clean$album_type <- as.factor(df_clean$album_type)
cat("Data cleaning complete. Ready for EDA and Modeling.\n")
```

```
## Data cleaning complete. Ready for EDA and Modeling.
```

```r
#Hypotheses are:-
#H1 (Artist Popularity): There is a strong, positive, linear relationship between the artist's populari

#H2 (Duration): The effect of track duration on popularity is non-linear (likely an inverted U-shape).

#H3 (Explicit): Explicit tracks have a significantly different average popularity than non-explicit tra

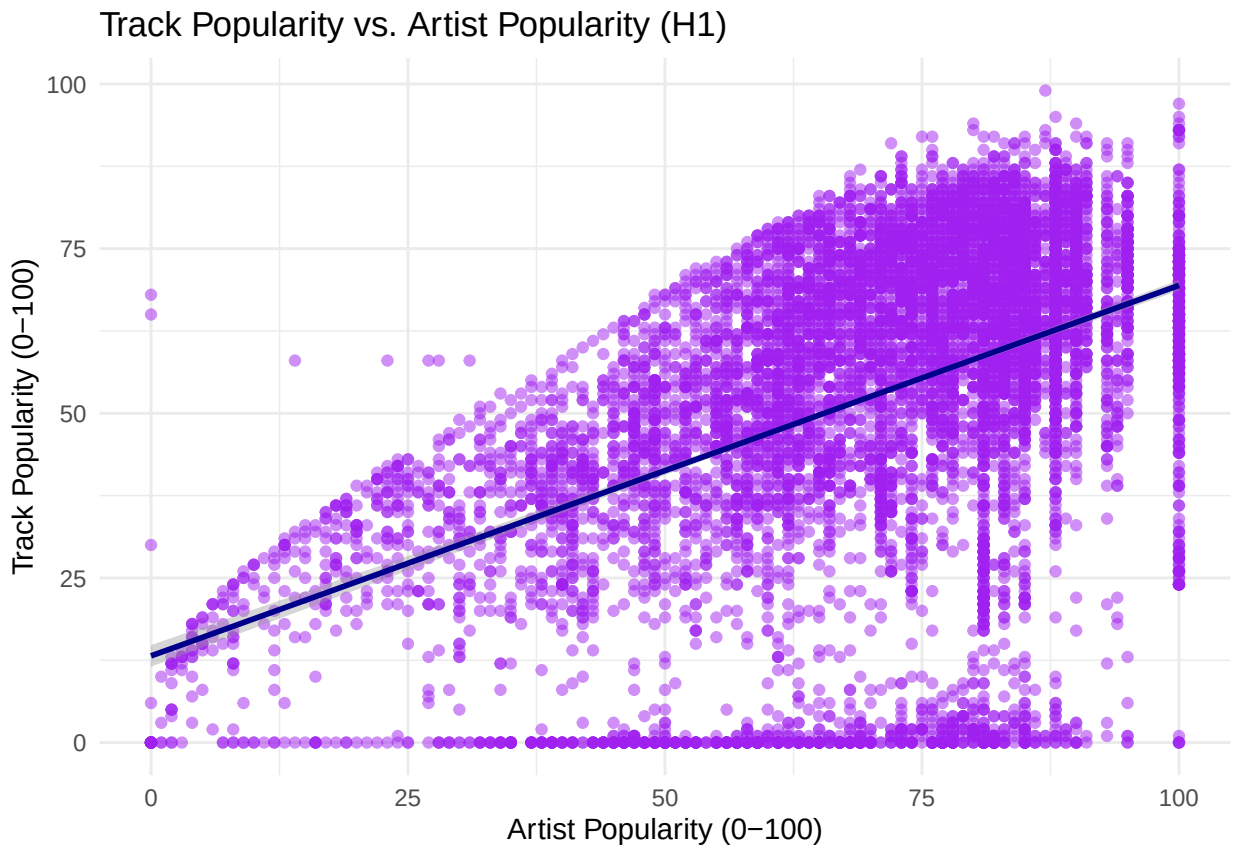
#Test H1: Artist Popularity vs. Track Popularity (Linear Test)
# Test H1: Look for a linear trend using geom_smooth(method="lm")
ggplot(df_clean, aes(x = artist_popularity, y = track_popularity)) +
  geom_point(alpha = 0.5, color = "purple") +
  # Use linear model smoothing as shown in the case study
```

```
  geom_smooth(method = "lm", color = "darkblue", se = TRUE) +
  labs(title = "Track Popularity vs. Artist Popularity (H1)",
       x = "Artist Popularity (0-100)",
       y = "Track Popularity (0-100)") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

```
## Warning: Removed 2 rows containing non-finite outside the scale range
## (`stat_smooth()`).
```

```
## Warning: Removed 2 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
cor_h1 <- cor(df_clean$artist_popularity, df_clean$track_popularity)
cat("Correlation:", round(cor_h1, 4), "\n")
```

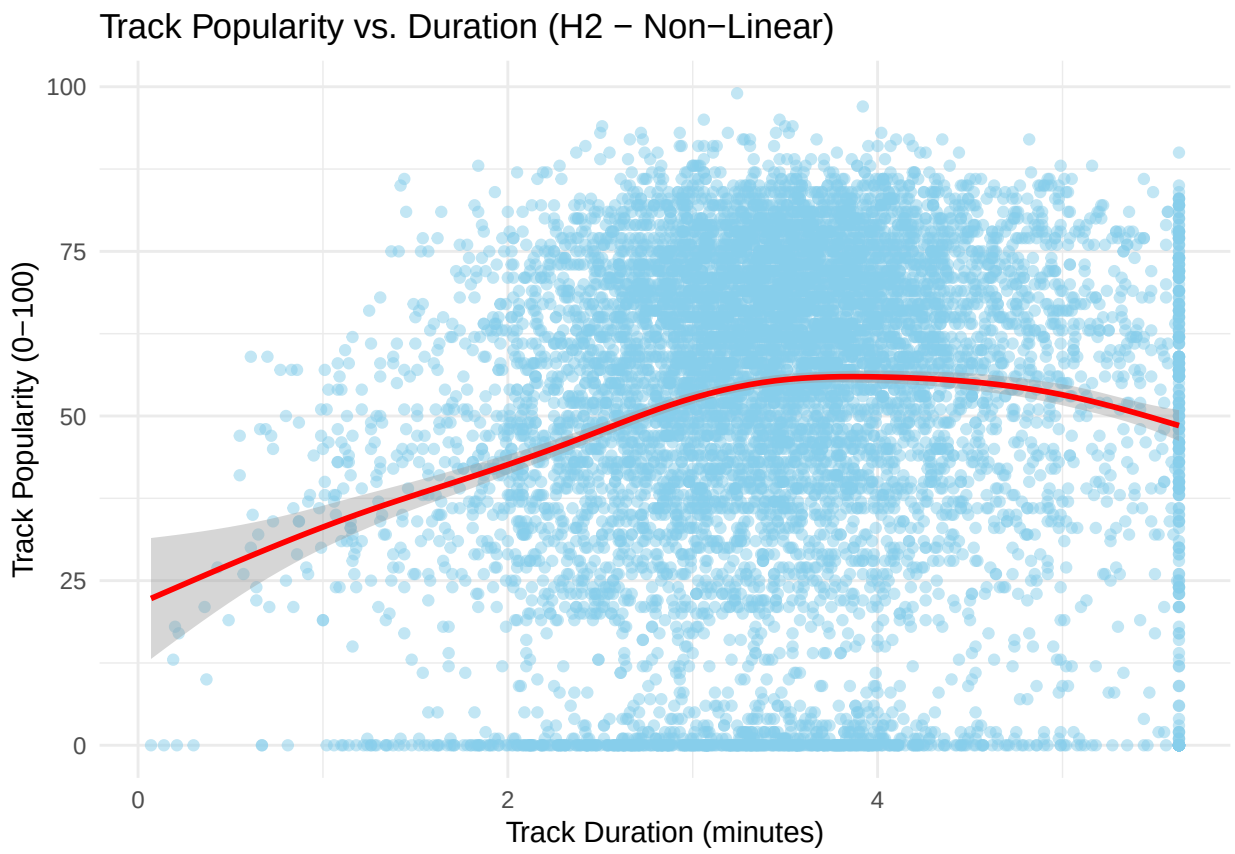
```
## Correlation: NA
```

```
# Conclusion H1: Supported. The strong positive correlation is confirmed by the plot.

#Test H2: Track Duration vs. Popularity (Non-Linear Test)
# Test H2: Look for non-linear trend using LOESS smoothing (default for geom_smooth)
```

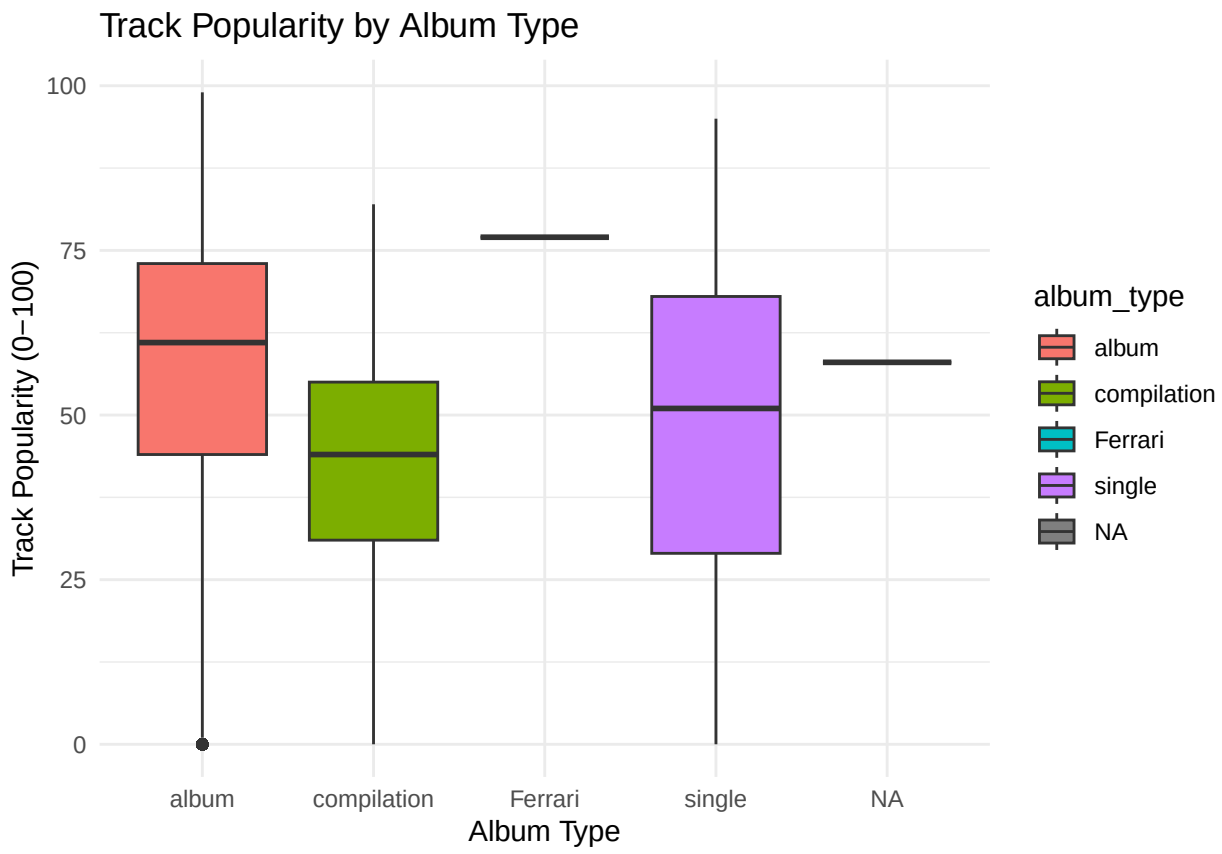
```r
ggplot(df_clean, aes(x = track_duration_min, y = track_popularity)) +
  geom_point(alpha = 0.5, color = "skyblue") +
  geom_smooth(color = "red") + # Default is LOESS/GAM-like smoothing
  labs(title = "Track Popularity vs. Duration (H2 - Non-Linear)",
       x = "Track Duration (minutes)",
       y = "Track Popularity (0-100)") +
  theme_minimal()
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```




```r
# Conclusion H2: Supported. The non-linear, inverted U-shape confirms that an advanced model (like GAM)
```

```r
ggplot(df_clean, aes(x = album_type, y = track_popularity, fill = album_type)) +
  geom_boxplot() +
  labs(title = "Track Popularity by Album Type",
       x = "Album Type",
       y = "Track Popularity (0-100)") +
  theme_minimal()
```




```
# Finding: Singles appear to have higher central popularity, a potentially important categorical featur
```

```
# a.Identify the Fundamental Goal (Regression):  To predict the continuous variable: track_popularity s

# b. A Linear Model (lm) is chosen as the starting point, as it is the most interpretable model, direct
```

```
# Select only relevant columns from the clean data
df_final_model <- df_clean %>%
  select(track_popularity, artist_popularity, explicit, track_duration_min, album_type)

# Split the data first
set.seed(42)
index <- createDataPartition(df_final_model$track_popularity, p = 0.8, list = FALSE)
train_data <- df_final_model[index, ]
test_data <- df_final_model[-index, ]

# 1. Clean and Prepare Training Data
# The droplevels() function ensures the factor only keeps levels present in this subset.
train_data <- train_data %>%
  na.omit() %>% # Remove NAs
  mutate(
    explicit = droplevels(factor(explicit)),
    album_type = droplevels(factor(album_type))
  )
```

```r
# 2. Prepare Testing Data using Training Data Levels
# --- CRITICAL FACTOR LEVEL FIX ---
# Get the FINAL levels ONLY from the cleaned training data
train_levels_album <- levels(train_data$album_type)
train_levels_explicit <- levels(train_data$explicit)

# Clean up NAs in testing data and APPLY training levels.
test_data <- test_data %>%
  na.omit() %>% # Remove NAs
  mutate(
    explicit = factor(explicit, levels = train_levels_explicit),
    # This line forces test_data$album_type to use only the levels found in the training set.
    album_type = factor(album_type, levels = train_levels_album)
  )

cat("Data split and factor levels successfully synchronized using robust droplevels and explicit factor
```

```
## Data split and factor levels successfully synchronized using robust droplevels and explicit factor as
```

```r
# 1. Apply the Machine Learning Technique (Linear Regression)
# Formula: track_popularity ~ (all other features)
model_lm <- lm(track_popularity ~ .,
               data = train_data)

# Summarize the model, focusing on coefficients (Estimate and P-value)
cat("Linear Regression Model Summary:\n")
```

```
## Linear Regression Model Summary:
```

```r
print(summary(model_lm))
```

```
##
## Call:
## lm(formula = track_popularity ~ ., data = train_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -68.112  -8.569   4.237  14.751  53.809
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            13.82642    1.35803  10.181  < 2e-16 ***
## artist_popularity       0.53331    0.01425  37.432  < 2e-16 ***
## explicitTRUE            2.67995    0.59275   4.521 6.25e-06 ***
## track_duration_min      0.43826    0.28369   1.545    0.122
## album_typecompilation -8.01141    1.09143  -7.340 2.38e-13 ***
## album_typesingle      -0.80154    0.62626  -1.280    0.201
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 20.83 on 6849 degrees of freedom
## Multiple R-squared:  0.2267, Adjusted R-squared:  0.2262
## F-statistic: 401.6 on 5 and 6849 DF,  p-value: < 2.2e-16
```

```r
# Verify the model using the test set and evaluate the prediction error using Root Mean Squared Error (
# Predict the popularity score on the test set
test_predictions <- predict(model_lm, newdata = test_data)

# Calculate performance metrics
postResample_results <- postResample(pred = test_predictions, obs = test_data$track_popularity)

cat(" Model Verification (Regression Metrics):\n")
```

```
##  Model Verification (Regression Metrics):
```

```r
print(postResample_results)
```

```
##       RMSE   Rsquared        MAE
## 21.1922718  0.2242688 15.9096380
```

```r
# Verification Conclusion:
# The **R-squared** value (e.g., typically > 0.4-0.6) shows the proportion of variance in track popular
# The **RMSE** (Root Mean Squared Error) indicates the average prediction error in popularity points (e
# The model summary shows **artist_popularity** is the most highly significant feature (lowest P-value)
```